

Today 12:12 PM

Lab\_4\_VibeCoding...  
PDFVibeCodingGitRep...  
Spreadsheet[https://github.com/SETAPESU26/11\\_maze\\_chase](https://github.com/SETAPESU26/11_maze_chase)

This is my vibe coding git repo. Please give me steps one-by-one

## Step 1 — Open and understand your assigned repo

Your repo is **SETAPESU26/11\_maze\_chase** . I checked its README. It is a Python/Pygame maze-chase game where the enemy uses BFS pathfinding.  GitHub

The lab handout says the first things to do are: go to the assigned repo, read the README, clone/fork it, and run the Python game.  Lab\_4\_VibeCoding\_Student\_handout

## Your repo's required tasks

You eventually need to complete these **4 tasks**:

1. **Multiple Enemies** — add 2 more enemies.
2. **Speed Up Over Time** — enemy gets faster every 15 seconds.
3. **Power Pellet** — collecting it freezes the enemy for 5 seconds.
4. **Score by Distance/Survival** — increase and display survival score.  GitHub

But **don't modify anything yet**.

## Do this now

Open your repository:

[Your Maze Chase GitHub repository](#) ↗

Then make sure you can see:

⌕ Plain text

```
11_maze_chase/
├─ game/
├─ .gitignore
├─ README.md
├─ main.py
└─ requirements.txt
```

Your README specifically says the game is run with:

```
</> powershell
```

```
pip install -r requirements.txt
```

```
python main.py
```

```
``` :chatgpt-content-reference{index="4"}
```

```
### Important: before changing anything
```

The lab requires a **\*\*10-second video before modifications\*\*** and another 10-second video after all modifica

So **\*\*first we need to get the original game running\*\***.

If you already have the repo cloned on your PC, open PowerShell in that folder and run:

```
```powershell
```

```
pip install -r requirements.txt
```

Then:

```
</> powershell
```

```
python main.py
```

**Do not make any code changes yet.**

Once the game opens, **play the original game for a few seconds and record the required ~10-second "before" video.**

Then tell me **"done"** (or send me what you see if there is an error), and I'll give you **Step 2 only**.

I want steps on how to do the repo clone also... like complete steps from zero

Worked for 6s

Absolutely. Since you want the **complete process from zero**, we'll treat it as if you have nothing set up yet.

I'll guide you **one step at a time**. Do the current step, tell me the result, and then I'll give you the next step. That way we can catch errors immediately.

Your lab instructions require you to clone/fork the assigned repo, run the original game, record the **before** video, make the changes with prompts, use separate commits for each task, and finally push your work to your own repo.  Lab\_4\_VibeCoding\_Student\_handout

Your README has **4 tasks**:

1. Multiple Enemies
2. Speed Up Over Time
3. Power Pellet
4. Score by Distance/Survival

---

## STEP 0 — What we're going to do

We'll follow this exact sequence:

 Plain text

0. Check/install Git
1. Open GitHub and verify the assigned repo
2. Fork the repo to your own GitHub account
3. Clone your fork to your PC
4. Open the project in VS Code
5. Create Python virtual environment
6. Install requirements
7. Run the ORIGINAL game
8. Record 10-second BEFORE video
9. Understand the code
10. Task 1 → prompt → test → commit → push
11. Task 2 → prompt → test → commit → push
12. Task 3 → prompt → test → commit → push
13. Task 4 → prompt → test → commit → push
14. Record 10-second AFTER video
15. Export ChatGPT conversation
16. Create Lab-4 folder/repository submission

**Important:** Don't modify the code yet.

---

## STEP 1 — Check whether Git is installed

Since you're on Windows, open **PowerShell**.

You can do this by:

**Windows key → type PowerShell → open Windows PowerShell**

Then type:

```
</> powershell
```

```
git --version
```

**If you get something like:**

```
</> Plain text
```

```
git version 2.51.0.windows.1
```

Git is already installed.

**If you get:**

```
</> Plain text
```

```
git : The term 'git' is not recognized...
```

then Git isn't installed and we'll install it before continuing.

---

## Your task right now

Open PowerShell and run:

```
</> powershell
```

```
git --version
```

**Send me exactly what PowerShell gives you.**

Don't do anything else yet. Once you send that output, I'll give you **STEP 2 — Fork your assigned**

repo to your own GitHub account, including exactly where to click.

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4> git --  
version  
git version 2.55.0.windows.4  
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4>
```

Perfect. Git is installed and working:

Plain text

```
git version 2.55.0.windows.4
```

So we can continue.

## STEP 2 — Fork the assigned repository

The lab specifically says to **clone or fork the assigned repo**, and you must work in your personal repository rather than raising a PR to the SETAPESU26 main repo.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

We will use **Fork**, because this gives you your own copy on GitHub.

### 2.1 Open the assigned repository

Go to:

[11\\_maze\\_chase repository](#)

You should see the repository belonging to:

Plain text

```
SETAPESU26
```

and the repository name:

Plain text

```
11_maze_chase
```

### 2.2 Click Fork

On the GitHub repository page, look at the **top-right area**.

Click:

**Fork**

GitHub will open a page asking where you want to fork it.

## 2.3 Select your account

Choose **your personal GitHub account**.

Your resulting repository should look like:

</> Plain text

```
YOUR_GITHUB_USERNAME/11_maze_chase
```

**NOT:**

</> Plain text

```
SETAPESU26/11_maze_chase
```

For example, if your GitHub username were `abc123` , it would be:

</> Plain text

```
abc123/11_maze_chase
```

## 2.4 Create the fork

Click:

**Create fork**

Wait until GitHub finishes creating it.

---

## STOP HERE

Don't clone anything yet.

After the fork is created, look at the URL in your browser.

It should be something like:

</> Plain text

```
https://github.com/YOUR_USERNAME/11_maze_chase
```

Send me the **new fork URL**.

Then I'll give you **STEP 3 — Clone your fork to this exact Windows folder**:

</> Plain text

```
C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4
```

and we'll do the Git commands one-by-one.

 [https://github.com/PES1UG24AM311/11\\_maze\\_chase-SE\\_Lab---4-](https://github.com/PES1UG24AM311/11_maze_chase-SE_Lab---4-)

Worked for a couple of seconds

Perfect. I verified that your fork exists and its default branch is `main`.

Your repository is now:

</> Plain text

```
PES1UG24AM311/11_maze_chase-SE_Lab---4-
```

## STEP 3 — Clone your fork to your computer

You are currently here in PowerShell:

</> Plain text

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4>
```

That is exactly the folder where we want the project.

### 3.1 Make sure you are in the correct folder

Run:

```
</> powershell
```

```
pwd
```

You should get:

```
</> Plain text
```

```
Path
```

```
----
```

```
C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4
```

### 3.2 Clone your repository

Now copy and run this **exact command**:

```
</> powershell
```

```
git clone https://github.com/PES1UG24AM311/11_maze_chase-SE_Lab---4-.git
```

You should see something similar to:

```
</> Plain text
```

```
Cloning into '11_maze_chase-SE_Lab---4-'...
```

```
remote: Enumerating objects...
```

```
remote: Counting objects...
```

```
Receiving objects...
```

```
Resolving deltas...
```

### 3.3 Enter the cloned project

After cloning finishes, run:

```
</> powershell
```

```
cd "11_maze_chase-SE_Lab---4-"
```

Your prompt should then become approximately:

</> Plain text

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11_maze_chase-SE_Lab---4->
```

### 3.4 Check that Git is connected correctly

Run:

</> powershell

```
git remote -v
```

You should see:

</> Plain text

```
origin https://github.com/PES1UG24AM311/11_maze_chase-SE_Lab---4-.git (fetch)
origin https://github.com/PES1UG24AM311/11_maze_chase-SE_Lab---4-.git (push)
```

This is **important** because `origin` should point to **your fork**, not `SETAPESU26/11_maze_chase` .

### 3.5 Check the files

Run:

</> powershell

```
dir
```

You should see files/folders such as:

</> Plain text

```
game
main.py
README.md
requirements.txt
```

Your README specifies that the project is run with `pip install -r requirements.txt` followed by `python main.py` .  Lab\_4\_VibeCoding\_Student\_handout

---

## STOP HERE

Run these commands **in order**:

```
</> powershell
```

```
pwd
```

```
</> powershell
```

```
git clone https://github.com/PES1UG24AM311/11_maze_chase-SE_Lab---4-.git
```

```
</> powershell
```

```
cd "11_maze_chase-SE_Lab---4-"
```

```
</> powershell
```

```
git remote -v
```

```
</> powershell
```

```
dir
```

Send me the output of those commands, especially `git remote -v` .

Then we'll do **STEP 4 — Open the cloned project in VS Code and set up Python from scratch.**

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4> git
clone https://github.com/PES1UG24AM311/11_maze_chase-
SE_Lab---4-.git
Cloning into '11_maze_chase-SE_Lab---4-'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
```

```
remote: Compressing objects: 100% (8/8), done.
remote: Total 11 (delta 0), reused 11 (delta 0), pack-reused 0 (from
0)
Receiving objects: 100% (11/11), done.
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4> cd
"11_maze_chase-SE_Lab---4-"
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-> git remote -v
origin https://github.com/PES1UG24AM311/11_maze_chase-
SE_Lab---4-.git (fetch)
origin https://github.com/PES1UG24AM311/11_maze_chase-
SE_Lab---4-.git (push)
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-> dir
```

```
Directory: C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab
- 4\11_maze_chase-SE_Lab---4-
```

| Mode   | LastWriteTime      | Length | Name             |
|--------|--------------------|--------|------------------|
| d----- | 01-Oct-26 12:16 PM |        | game             |
| -a---- | 01-Oct-26 12:16 PM | 34     | .gitignore       |
| -a---- | 01-Oct-26 12:16 PM | 116    | main.py          |
| -a---- | 01-Oct-26 12:16 PM | 2339   | README.md        |
| -a---- | 01-Oct-26 12:16 PM | 15     | requirements.txt |

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4->
```

Perfect — Step 3 is completely successful. 

You have now:

- Cloned your personal fork

- Entered the project directory
- Confirmed `origin` points to **your repository**
- Confirmed the project files are present

Your setup is currently:

⟨/⟩ Plain text

```
C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\  
└─ 11_maze_chase-SE_Lab---4-  
    ├── game\  
    ├── .gitignore  
    ├── main.py  
    ├── README.md  
    └─ requirements.txt
```

And importantly:

⟨/⟩ Plain text

```
origin → PES1UG24AM311/11_maze_chase-SE_Lab---4-
```

So don't change the remote.

---

## STEP 4 — Open the project in VS Code

We're going to open the **entire project folder**, not individual files.

You are currently inside:

⟨/⟩ Plain text

```
C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11_maze_chase-SE_Lab---4-
```

### 4.1 Check whether VS Code is available

In the same PowerShell window, run:

⟨/⟩ powershell

```
code --version
```

## If you get something like:

```
</> Plain text
```

```
1.xx.x
```

```
...
```

VS Code is installed and the `code` command works.

Then run:

```
</> powershell
```

```
code .
```

The `.` means:

Open the current directory in VS Code.

VS Code should open with:

```
</> Plain text
```

```
11_maze_chase-SE_Lab---4-
```

as the project/workspace.

---

## If `code --version` gives an error

Don't install anything yet.

Just send me the error and I'll guide you through it.

---

## STOP HERE

Run only:

```
</> powershell
```

```
code --version
```

Send me the output.

Then I'll give you **STEP 5 — Create the Python virtual environment ( .venv )**.

Opened vs code

Great. 

## STEP 5 — Create a Python virtual environment

We want the project's Python packages isolated from the rest of your computer.

### 5.1 Open the VS Code terminal

In VS Code:

**Terminal → New Terminal**

A terminal should appear at the bottom.

Make sure its path ends with:

```
</> Plain text
```

```
11_maze_chase-SE_Lab---4->
```

For example:

```
</> Plain text
```

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11_maze_chase-SE_Lab---4->
```

### 5.2 Check Python

Run:

```
</> powershell
```

```
python --version
```

You should get something like:

</> Plain text

```
Python 3.11.x
```

## 5.3 Create the virtual environment

Then run:

</> powershell

```
python -m venv .venv
```

This creates:

</> Plain text

```
11_maze_chase-SE_Lab---4-  
├─ .venv  
├─ game  
├─ main.py  
├─ README.md  
└─ requirements.txt
```

## 5.4 Activate it

Run:

</> powershell

```
.\.venv\Scripts\Activate.ps1
```

If successful, your terminal should change from:

</> Plain text

```
PS C:\...\11_maze_chase-SE_Lab---4->
```

to something like:

```
</> Plain text
```

```
(.venv) PS C:\...\11_maze_chase-SE_Lab---4->
```

The `(.venv)` at the beginning is what we want.

---

### ⚠ If PowerShell gives an execution-policy error

Don't try random commands.

You may see something like:

```
</> Plain text
```

```
running scripts is disabled on this system
```

If that happens, **send me the exact error** and I'll give you the appropriate fix.

---

## STOP HERE

Run these **three commands in order**:

```
</> powershell
```

```
python --version
```

```
</> powershell
```

```
python -m venv .venv
```

```
</> powershell
```

```
.\.venv\Scripts\Activate.ps1
```

Then send me what you get.

Don't install requirements or run the game yet. We'll do those in the next step.

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-> .\venv\Scripts\Activate.ps1
.\venv\Scripts\Activate.ps1 : File C:\Personal\PES College
Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-\venv\Scripts\Activate.ps1 cannot
be loaded because running scripts is disabled on this system.
For more information, see about_Execution_Policies at https:/
go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ .\venv\Scripts\Activate.ps1
+ ~~~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

That's the exact PowerShell execution-policy issue we expected. Your `.venv` was created successfully; only **activation** was blocked.

We'll fix this without changing the system-wide policy.

## STEP 5A — Allow local PowerShell scripts for your user

In the **same VS Code PowerShell terminal**, run:

```
</> powershell
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

PowerShell may ask:

```
</> Plain text
Execution Policy Change
...
[Y] Yes [A] Yes to All [N] No ...
```

Type:

</> Plain text

Y

and press **Enter**.

**Then activate the virtual environment again**

Run:

</> powershell

```
.\.venv\Scripts\Activate.ps1
```

If successful, your terminal should now begin with:

</> Plain text

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11_maze_chase-SE_Lab---4->
```

The important part is:

</> Plain text

```
(.venv)
```

---

**Don't run anything else yet**

Once you see `(.venv)` , send me the terminal output.

Then we'll move to **STEP 6 — Install the project's requirements and verify Pygame**, before running the original game.

I want to allow local powerscript only for that session... I don't want risk

Yes — that's a better choice if you only want the permission for this **PowerShell session**. We don't need to change your user or system execution policy.

**STEP 5A — Session-only execution policy**

In the current VS Code PowerShell terminal, run:

```
</> powershell  
  
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process
```

If it asks for confirmation, type:

```
</> Plain text  
  
Y
```

## Why this is safer for your requirement

-Scope Process means the change applies **only to the current PowerShell process**.

So:

- ☒ It allows your `.venv` activation now.
- ☒ It does **not** change the system execution policy.
- ☒ It does **not** change your user-level execution policy.
- ☒ Closing this VS Code terminal/PowerShell session removes the change.
- ☒ A new PowerShell session goes back to its previous policy.

Then activate your environment:

```
</> powershell  
  
.\.venv\Scripts\Activate.ps1
```

You should see:

```
</> Plain text  
  
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11_maze_chase-SE_Lab---4->
```

**Do only those two commands for now.** Send me what you see after activation, and we'll continue to Step 6.

done

Perfect.  Your virtual environment is activated for this session.

## STEP 6 — Install the project requirements

You should currently see something like:

 Plain text

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11_maze_chase-SE_Lab---4->
```

Now run:

 powershell

```
pip install -r requirements.txt
```

This reads the project's `requirements.txt` and installs the packages needed by the game.

After it finishes, **don't run the game yet.**

Then run:

 powershell

```
pip list
```

You should see the installed packages, including **pygame**.

### What we're checking

Your repository's README gives the setup commands as:

 Plain text

```
pip install -r requirements.txt  
python main.py
```

So we're following the repository's own instructions rather than changing the project setup.

 Lab\_4\_VibeCoding\_Student\_handout

## Do this now

Run:

```
</> powershell  
  
pip install -r requirements.txt
```

Wait for it to finish.

Then:

```
</> powershell  
  
pip list
```

Send me the output (or any error you get).

**Don't run `python main.py` yet.** After we verify the installation, we'll do the next step: **run the completely original game and prepare your 10-second BEFORE recording.**

```
PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> .\venv\Scripts\Activate.ps1  
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> pip install -r requirements.txt  
Collecting pygame==2.5.2 (from -r requirements.txt (line 1))  
  Downloading pygame-2.5.2-cp311-cp311-  
win_amd64.whl.metadata (13 kB)  
  Downloading pygame-2.5.2-cp311-cp311-win_amd64.whl (10.8  
MB)  
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.8/10.8 MB 4.2  
MB/s eta 0:00:00  
Installing collected packages: pygame  
Successfully installed pygame-2.5.2  
  
[notice] A new release of pip is available: 24.0 -> 26.2.1  
[notice] To update, run: python.exe -m pip install --upgrade pip  
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> pip list
```

| Package    | Version |
|------------|---------|
| pip        | 24.0    |
| pygame     | 2.5.2   |
| setuptools | 65.5.0  |

```
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4->
```

Perfect.  **STEP 6 is complete.**

You have:

 Plain text

```
Python environment: .venv
pygame: 2.5.2
```

And `pygame==2.5.2` matches the project's `requirements.txt`, so the setup is correct.

The pip upgrade notice can be ignored for this lab. **Don't upgrade pip or pygame.** We want to keep the repository's specified environment unchanged.

## STEP 7 — Run the ORIGINAL game

This is important because we haven't modified anything yet.

In the same terminal, run:

 powershell

```
python main.py
```

### What should happen

A Pygame window should open with the original **Maze Chase** game.

According to the README, the controls are:

| Key           | Action  |
|---------------|---------|
| W / A / S / D | Move    |
| Arrow keys    | Move    |
| R             | Restart |

[Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

## For now, DON'T change anything

Just verify that the original game works.

Once the game opens:

1. Play it for a little while.
2. Observe the current behavior.
3. **Record approximately 10 seconds of gameplay.**
4. This will be your **BEFORE video**.

The lab specifically requires a 10-second recording before making changes and another after all changes.

[Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

## If the game doesn't open

If you get an error in the terminal, **don't try to fix it yourself yet**. Send me the complete error.

## If the game opens successfully

You can close the game after recording the BEFORE video.

Then tell me:

**Game works and before video recorded.**

We'll move to **STEP 8** — inspect the original code and prepare the first Vibe Coding prompt (Task 1: **Multiple Enemies**).

I recorded 'before' video

Excellent.  **STEP 7 is complete.**

You now have the required **BEFORE video** showing the original game. The lab specifically requires this before making any changes.  Lab\_4\_VibeCoding\_Student\_handout

## STEP 8 — Check the Git status before making changes

Before we start Vibe Coding, we want to establish a clean baseline.

In the VS Code terminal, make sure you still see:

```
</> Plain text
```

```
(.venv)
```

Then run:

```
</> powershell
```

```
git status
```

You should ideally see something like:

```
</> Plain text
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

This is important because from this point onward, **each lab task needs its own separate commit.**

 Lab\_4\_VibeCoding\_Student\_handout

**Don't modify anything yet.**

Run only:

```
</> powershell
```

```
git status
```

Send me the output.

After that, we'll do **STEP 9** — inspect the original code and make your first Vibe Coding change:

### Task 1, Multiple Enemies.

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean  
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4->
```

Perfect.  **STEP 8 is complete.**

Your baseline is clean:

 Plain text

```
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

That means we're ready to start modifying the project.

## STEP 9 — Understand Task 1 before prompting

Your first required feature is:

### Task 1: Multiple Enemies

The README says:

- Add **2 more enemies**
- They should start at **different corners**
- Each enemy should **independently chase the player**
- Use a `self.enemies` list and loop over the enemies
- Each should independently call `bfs()` during updates

The repository even provides a suggested Vibe Coding prompt for this task. We should use that as our starting prompt rather than inventing a different requirement.

## Your first Vibe Coding prompt

Open **ChatGPT** (this conversation is fine).

Send the following prompt:

Task 1 Vibe Coding Prompt



In my maze-chase pygame game, I have one Enemy that uses BFS to chase the player. Add two more Enemy instances starting at different maze corners. Each should independently call bfs() on every update. List them in a self.enemies list and loop over them.

This directly corresponds to **Task 1** in your repository README.

---

## IMPORTANT — Don't ask ChatGPT to do all 4 tasks at once

For this lab, we'll do:

Plain text

Task 1



Vibe Coding prompt



Review changes



Run/test game



Commit Task 1



Push Task 1



Task 2



...

This will give you the **separate commits** for each task that your lab requires.



Lab\_4\_VibeCoding\_Student\_handout

## One more important thing

**Do not copy-paste code manually yet.**

Send that prompt to ChatGPT and let the Vibe Coding process make the changes in your project.

Since we're already in this ChatGPT conversation, you can also simply give me that prompt and I can help you work through the code changes.

---

## STOP HERE

For now, **send the Task 1 prompt above to your Vibe Coding tool.**

Once it makes changes, **don't commit yet.**

Send me what it changed / the response it gave you, and I'll walk you through **reviewing and testing Task 1 before committing it.**

I mean you are chatgpt right? so you can also do it right?

Worked for 19s

Yes — I can do the Vibe Coding part with you. I inspected the actual code, and Task 1 is straightforward.

However, I **cannot directly push the code into your GitHub fork from this connection** because the GitHub integration has read access but the write operation was denied. I won't pretend I changed your repo.

So we'll do the actual Vibe Coding change **locally in your VS Code**, which is also better for your lab because you need to review, test, commit, and push the changes yourself.

## STEP 9A — Let me give you the Task 1 change

Open this file in VS Code:

⟨/⟩ Plain text

```
game
└─ game_engine.py
```

Find this line in `reset()` :

Python

```
self.enemy = Enemy(ROWS-1, COLS-1)
```

Replace it with:

Python

```
self.enemies = [  
    Enemy(ROWS-1, COLS-1), # bottom-right  
    Enemy(ROWS-1, 0),     # bottom-left  
    Enemy(0, COLS-1),      # top-right  
]
```

Then find this in `update()` :

Python

```
self.enemy.update(self.walls, self.player, ROWS, COLS)  
if self.player.rect.colliderect(self.enemy.rect):  
    self.caught = True
```

Replace it with:

Python

```
for enemy in self.enemies:  
    enemy.update(self.walls, self.player, ROWS, COLS)  
    if self.player.rect.colliderect(enemy.rect):  
        self.caught = True
```

Finally, in `draw()` , find:

Python

```
self.enemy.draw(self.screen)
```

Replace it with:

Python

```
for enemy in self.enemies:  
    enemy.draw(self.screen)
```

## What Task 1 now does

You will have three enemies:

</> Plain text

```
Enemy 1 → bottom-right  
Enemy 2 → bottom-left  
Enemy 3 → top-right
```

and every game update does:

</> Plain text

```
for each enemy  
    ↓  
    enemy.update()  
    ↓  
    BFS independently finds path  
    ↓  
    check collision with player
```

This matches the Task 1 requirement from your repository README.

## Important

**Don't change anything else.**

After making those three changes, **save `game_engine.py`** .

Then tell me:

**Task 1 changes saved**

I'll give you the next step: **run and test Task 1**, before we make the Task 1 Git commit.

I did

Great.  **Task 1 changes are saved locally.**

## STEP 10 — Test Task 1

We are **not committing yet**. First we need to prove that the feature works.

In the VS Code terminal, make sure you still see:

```
</> Plain text
```

```
(.venv)
```

Then run:

```
</> powershell
```

```
python main.py
```

### What you should check

The game should now have **3 enemies** instead of 1.

You should see enemies starting from different corners:

- Bottom-right
- Bottom-left
- Top-right

Move your player around and verify that **all three enemies move toward you independently**.

Also check that the game still behaves correctly:

- W/A/S/D or arrow keys move the player.
- If any enemy catches you, you get **CAUGHT!** .
- Reaching the exit still gives **ESCAPED!** .
- **R** still restarts the game.

### Don't worry if the enemies don't move simultaneously

The `Enemy` class has:

```
</> Python
```

```
self.move_interval = 20
```

so each enemy updates its BFS movement periodically rather than every rendered frame. That's the existing behavior.

---

## After testing

Close the game and tell me one of these:

**If it works:**

Task 1 works — I see 3 enemies.

**If something is wrong:**

Task 1 has an error

and paste the error or describe what happened.

**Don't run `git commit` yet.**

Once we've confirmed Task 1 works, I'll walk you through:

⌄ Plain text

```
git status
↓
review exactly what changed
↓
git diff
↓
git add
↓
git commit
↓
git push
```

That will give you the **separate Task 1 commit** required by your lab.

 Lab\_4\_VibeCoding\_Student\_handout

File "C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11\_maze\_chase-SE\_Lab---4-\main.py", line 5, in <module>

```

engine.run()
File "C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-\game\game_engine.py", line 90, in
run
    self.draw()
File "C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-\game\game_engine.py", line 65, in
draw
    self.enemy.draw(self.screen)
    ^^^^^^^^^^^^^
AttributeError: 'GameEngine' object has no attribute 'enemy'

```

Good — this is a **simple incomplete replacement**, not a problem with the BFS or the new enemies.

The error tells us exactly what happened:

</> Plain text

```
AttributeError: 'GameEngine' object has no attribute 'enemy'
```

We changed `self.enemy` into `self.enemies` , but one old reference remains in `draw()` .

## STEP 10A — Fix the remaining reference

Open:

</> Plain text

```
game/game_engine.py
```

Go to approximately **line 65**.

You still have:

</> Python

```
self.enemy.draw(self.screen)
```

Replace that line with:

</> Python

```
for enemy in self.enemies:
    enemy.draw(self.screen)
```

So that section should look like:

</> Python

```
self.player.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
hud=pygame.Rect(0,ROWS*CELL,WIDTH,50)
```

## Save the file

Press:

</> Plain text

Ctrl + S

---

## STEP 10B — Run it again

In the terminal:

</> powershell

```
python main.py
```

Now check whether you can see **three enemies**.

If it works, close the game and tell me:

### Task 1 works now

If another error appears, send me the complete error.

**Don't commit yet.** We're going to make sure Task 1 is fully working before creating the separate Git commit.

WASD wasn't working but 3 enemies were there

Good catch. Since **three enemies are appearing**, our Task 1 changes are working. The WASD issue is separate and we should fix it before committing.

The original `Player.move()` code already supports both WASD and arrow keys, so we should **not** rewrite the movement system unnecessarily.

## STEP 10C — Test the arrow keys

First, run the game again:

```
</> powershell  
  
python main.py
```

Try:

- ↑
- ↓
- ←
- →

Tell me whether the **arrow keys move the player**.

Don't change any code yet. This will tell us whether the problem is specifically with keyboard input or something else.

nope... not working

That tells us something important: **Task 1 itself isn't responsible for the movement code**, because we only changed enemy handling. The original `Player.move()` already contains both WASD and arrow-key handling.

Let's diagnose this before changing anything.

## STEP 10D — Check whether the game window has keyboard focus

Run:

```
</> powershell
```

```
python main.py
```

When the game window appears:

1. Click once inside the game window.
2. Then hold `W` for 1–2 seconds.
3. Try `A`, `S`, and `D`.
4. Try the arrow keys.

Also check whether the player is actually visible as the **blue circle**.

### If it still doesn't move

Close the game with the window's **X**.

Then run this command in the terminal:

```
</> powershell
```

```
python -c "import pygame; pygame.init(); print(pygame.get_init()); print(pygame.key.get_focused()); pygame"
```

Send me the output.

**Don't modify the code yet.** We want to identify whether this is a keyboard-focus/Pygame issue or something introduced by our change before touching Task 1 further.

ok wait I'm not even able to see myself, i.e., the ball

Ah — that changes the diagnosis. If you **can't even see the blue player**, then this isn't a WASD problem. We should stop testing movement and inspect what actually changed.

The original code draws the player with:

```
</> Python
```

```
self.player.draw(self.screen)
```

and our Task 1 change was not supposed to touch the player at all.

### STEP 10E — Inspect exactly what changed

Close the game.

```
</> powershell  
git diff
```

## Don't change anything.

I want to check whether something else was accidentally changed while editing `game_engine.py`.

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-> git diff
diff --git a/game/game_engine.py b/game/game_engine.py
index 00ccf9a..2697abe 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -20,7 +20,11 @@ class GameEngine:
    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
-       self.enemy = Enemy(ROWS-1, COLS-1)
+       self.enemies = [
+           Enemy(ROWS-1, COLS-1), # bottom-right
+           Enemy(ROWS-1, 0),      # bottom-left
+           ]
```

I need to see the **whole diff** before we change anything.

## STEP 10F — Print the full diff without the pager

36 of 101

Plain text

Q

if you're still inside the `git diff` screen.

Then run:

powershell

```
git --no-pager diff
```

That will print the complete changes directly in the terminal.

Copy everything from:

Plain text

```
diff --git a/game/game_engine.py...
```

all the way to the end and send it here.

I already notice the indentation in the snippet looks unusual:

Python

```
self.enemies = [  
    Enemy(...)
```

but let's see the complete file diff first rather than guessing.

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git --no-pager diff  
diff --git a/game/game_engine.py b/game/game_engine.py  
index 00ccf9a..2697abe 100644  
--- a/game/game_engine.py  
+++ b/game/game_engine.py  
@@ -20,7 +20,11 @@ class GameEngine:  
     def reset(self):  
         self.walls = generate_maze(COLS, ROWS)
```

```

        self.player = Player(0, 0)
-       self.enemy = Enemy(ROWS-1, COLS-1)
+       self.enemies = [
+       Enemy(ROWS-1, COLS-1), # bottom-right
+       Enemy(ROWS-1, 0),      # bottom-left
+       Enemy(0, COLS-1),      # top-right
+   ]

        self.exit_rect = pygame.Rect((COLS//2)*CELL+5,
(ROWS//2)*CELL+5, CELL-10, CELL-10)
        self.caught = False
        self.won = False
@@ -35,9 +39,10 @@ class GameEngine:
    if self.caught or self.won: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
-   self.enemy.update(self.walls, self.player, ROWS, COLS)
-   if self.player.rect.colliderect(self.enemy.rect):
-       self.caught = True
+   for enemy in self.enemies:
+       enemy.update(self.walls, self.player, ROWS, COLS)
+       if self.player.rect.colliderect(enemy.rect):
+           self.caught = True
    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

@@ -55,8 +60,10 @@ class GameEngine:
    pygame.draw.rect(self.screen,
(80,200,80),self.exit_rect,border_radius=4)
    lbl=self.font.render("EXIT",True,(20,80,20))
    self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
-   self.player.draw(self.screen)
-   self.enemy.draw(self.screen)
+   for enemy in self.enemies:
+       enemy.draw(self.screen)
+   for enemy in self.enemies:
+       enemy.draw(self.screen)

```

```
        hud=pygame.Rect(0,ROWS*CELL,WIDTH,50)
        pygame.draw.rect(self.screen,(30,30,50),hud)
        info=self.font.render("Reach EXIT before the enemy catches
you! R=Restart",True,(200,200,200))
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4->
```

Yes — we found the problem. There are **two mistakes** in the Task 1 edit.

### Problem 1 — You accidentally removed the player drawing

Your original code had:

Python

```
self.player.draw(self.screen)
self.enemy.draw(self.screen)
```

The new code has:

Python

```
for enemy in self.enemies:
    enemy.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
```

So the player is never drawn. That's why you couldn't see the blue ball.

### Problem 2 — Enemy drawing was duplicated

We only need **one** enemy loop.

---

## STEP 10G — Fix the `draw()` section

Open:

Plain text

```
game/game_engine.py
```

Find this:

```
</> Python
```

```
self.player.draw(self.screen)
```

It should have been preserved.

Your current section is:

```
</> Python
```

```
self.player.draw(self.screen)
```

**Wait — according to your diff, it has actually been replaced, so change the section to exactly this:**

```
</> Python
```

```
pygame.draw.rect(self.screen, (80,200,80),self.exit_rect,border_radius=4)
lbl=self.font.render("EXIT",True,(20,80,20))
self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
self.player.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
hud=pygame.Rect(0,ROWS*CELL,WIDTH,50)
```

Also, fix the indentation of your enemy list.

Change:

```
</> Python
```

```
self.enemies = [
Enemy(ROWS-1, COLS-1), # bottom-right
Enemy(ROWS-1, 0),      # bottom-left
Enemy(0, COLS-1),      # top-right
]
```

to:

Python

```
self.enemies = [  
    Enemy(ROWS-1, COLS-1), # bottom-right  
    Enemy(ROWS-1, 0),      # bottom-left  
    Enemy(0, COLS-1),      # top-right  
]
```

Python may technically accept the current indentation inside the brackets, but let's keep it clean and conventional.

---

## STEP 10H — Save and test again

Press:

Plain text

Ctrl + S

Then run:

powershell

python main.py

Now you should see:

-  **Blue player**
-  **Three enemies**
- Exit
- Maze

And the player should respond to:

Plain text

W A S D

and:

Plain text

↑ ↓ ← →

## Don't commit yet.

Test the game for a little while.

If everything works, tell me:

### Task 1 works now

Then we'll inspect the final `git diff` and create the **Task 1 commit only**.

```
def draw(self):
    self.screen.fill((230, 220, 210))
    wc=(50,40,60)
    for r in range(ROWS):
        for c in range(COLS):
            x,y=c*CELL,r*CELL
            w=self.walls[r][c]
            if w[0]: pygame.draw.line(self.screen,wc,(x,y),
(x+CELL,y),3)
            if w[1]: pygame.draw.line(self.screen,wc,(x,y+CELL),
(x+CELL,y+CELL),3)
            if w[2]: pygame.draw.line(self.screen,wc,(x+CELL,y),
(x+CELL,y+CELL),3)
            if w[3]: pygame.draw.line(self.screen,wc,(x,y),
(x,y+CELL),3)
        pygame.draw.rect(self.screen,
(80,200,80),self.exit_rect,border_radius=4)
        lbl=self.font.render("EXIT",True,(20,80,20))
        self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
        self.player.draw(self.screen)
        for enemy in self.enemies:
            enemy.draw(self.screen)
        hud=pygame.Rect(0,ROWS*CELL,WIDTH,50)
        pygame.draw.rect(self.screen,(30,30,50),hud)
        info=self.font.render("Reach EXIT before the enemy catches
```

```
you! R=Restart",True,(200,200,200))
self.screen.blit(info,(8,ROWS*CELL+14))
if self.caught:
    self._overlay("CAUGHT!", (220,60,60))
if self.won:
    self._overlay("ESCAPED!", (80,220,80))
pygame.display.flip()
```

Is this correct?

Yes — this `draw()` function is now correct for Task 1. 

The important part is:

 Python

```
self.player.draw(self.screen)

for enemy in self.enemies:
    enemy.draw(self.screen)
```

That means:

- The **blue player** is drawn again.
- All **three enemies** are drawn.
- There is only **one enemy-drawing loop**.
- The HUD and game-over overlays remain unchanged.

So your `draw()` function is good.

## One thing to verify

Your `reset()` should also have exactly:

 Python

```
self.enemies = [
    Enemy(ROWS-1, COLS-1), # bottom-right
    Enemy(ROWS-1, 0),      # bottom-left
    Enemy(0, COLS-1),      # top-right
]
```

And your `update()` should have:

`</>` Python

```
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
```

If those three sections are correct, **save the file and run:**

`</>` powershell

```
python main.py
```

You should now see the **blue player + 3 enemies**.

Don't commit yet. Test movement and enemy behavior first.

now task 1 is done I think

Yes. If you have verified that:

-  Player is visible
-  WASD/arrow keys work
-  Three enemies appear
-  All three enemies chase the player independently
-  Collision with any enemy causes **CAUGHT!**
-  Exit still works
- **R** still restarts the game

then **Task 1 is complete.** 

Before moving to Task 2, we need to make the required **separate Git commit for Task 1**. The lab explicitly requires separate commits for each task.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

## STEP 11 — Review Task 1 changes

First, close the game if it is still running.

Then in your VS Code terminal run:

```
</> powershell  
git --no-pager diff
```

We want to confirm that **only Task 1 changes** are present.

You should see changes in:

```
</> Plain text  
game/game_engine.py
```

and specifically:

```
</> Plain text  
self.enemy
```

changed to:

```
</> Plain text  
self.enemies
```

plus the loops for updating and drawing the enemies.

## Send me the output

Don't run `git add`, `git commit`, or `git push` yet.

Send me the `git --no-pager diff` output, and I'll verify it before we create the Task 1 commit.

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git --no-pager diff  
diff --git a/game/game_engine.py b/game/game_engine.py  
index 00ccf9a..cd15ae7 100644  
--- a/game/game_engine.py  
+++ b/game/game_engine.py  
@@ -20,7 +20,11 @@ class GameEngine:
```

```

def reset(self):
    self.walls = generate_maze(COLS, ROWS)
    self.player = Player(0, 0)
-   self.enemy = Enemy(ROWS-1, COLS-1)
+   self.enemies = [
+       Enemy(ROWS-1, COLS-1), # bottom-right
+       Enemy(ROWS-1, 0),      # bottom-left
+       Enemy(0, COLS-1),      # top-right
+   ]
    self.exit_rect = pygame.Rect((COLS//2)*CELL+5,
(ROWS//2)*CELL+5, CELL-10, CELL-10)
    self.caught = False
    self.won = False
@@ -35,9 +39,10 @@ class GameEngine:
    if self.caught or self.won: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
-   self.enemy.update(self.walls, self.player, ROWS, COLS)
-   if self.player.rect.colliderect(self.enemy.rect):
-       self.caught = True
+   for enemy in self.enemies:
+       enemy.update(self.walls, self.player, ROWS, COLS)
+       if self.player.rect.colliderect(enemy.rect):
+           self.caught = True
    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

@@ -56,7 +61,8 @@ class GameEngine:
    lbl=self.font.render("EXIT",True,(20,80,20))
    self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
    self.player.draw(self.screen)
-   self.enemy.draw(self.screen)
+   for enemy in self.enemies:
+       enemy.draw(self.screen)
    hud=pygame.Rect(0,ROWS*CELL,WIDTH,50)
    pygame.draw.rect(self.screen,(30,30,50),hud)

```

```
info=self.font.render("Reach EXIT before the enemy catches  
you! R=Restart",True,(200,200,200))  
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4->
```

Yes — this diff is **clean and correct for Task 1**. 

It contains exactly the intended changes:

1. `self.enemy` → `self.enemies` with 3 starting positions.
2. All enemies are updated independently.
3. Collision is checked against each enemy.
4. All enemies are drawn.
5. The player drawing remains intact.
6. No unrelated files were changed.

So now we can create the required **separate Task 1 commit**.

## STEP 12 — Commit Task 1

### 12.1 Stage the Task 1 file

Run:

```
</> powershell  
git add game/game_engine.py
```

### 12.2 Check what is staged

Run:

```
</> powershell  
git status
```

You should see something similar to:

```
</> Plain text
```

Changes to be committed:

modified: game/game\_engine.py

## 12.3 Create the Task 1 commit

Run:

```
</> powershell
```

```
git commit -m "Task 1: Add multiple enemies"
```

You should get something similar to:

```
</> Plain text
```

```
[main xxxxxxx] Task 1: Add multiple enemies
1 file changed, ...
```

## 12.4 Push Task 1 to your GitHub fork

Then run:

```
</> powershell
```

```
git push origin main
```

You should see something like:

```
</> Plain text
```

```
To https://github.com/PES1UG24AM311/11_maze_chase-SE_Lab---4-.git
... main -> main
```

This is the first separate commit required by your lab.



Lab\_4\_VibeCoding\_Student\_handout

---

## Do these one at a time

Start with:

```
</> powershell
```

```
git add game/game_engine.py
```

then:

```
</> powershell
```

```
git status
```

Stop there and send me the `git status` output.

I'll verify the staging before you commit.

I did it

Great. 👍

Now let's **commit Task 1** separately, as required by the lab.

## Step 1 — Commit Task 1

In the same VS Code terminal, run:

```
</> powershell
```

```
git commit -m "Task 1: Add multiple enemies"
```

Send me the **complete output** from that command.

**Don't push yet** — I'll verify the commit first, then we'll push it and move to **Task 2: Speed Up Over Time**.

- 1.Go to the repo assigned to you
- 2.Go through the ReadMe file to understand the deliverables
- 3.Clone or Fork the repo
- 4.Run the Python based code (A game/ A Puzzle)
- 5.Record 10 seconds of video before making any changes.
- 6.Write prompts and fix the broken code
- 7.Write prompts to add all the features listed in the ReadMe file of the repo one

by one.

8.Record another 10 seconds of video again after making all the changes.

9.Push the code to your repo.

10.Have Separate Commits for each task

11.Do not raise PR to the main repo (SETAPESU26)

Uhh.. I think we messed up

No — **we haven't messed up the project.** 😊 In fact, most of the workflow is still correct. The important thing is that we caught the checklist before going too far.

Let's compare what we actually did with the lab instructions:

| Lab requirement                           | Our status                              |
|-------------------------------------------|-----------------------------------------|
| 1. Go to assigned repo                    | ✅ Done                                  |
| 2. Read README                            | ✅ Done                                  |
| 3. Clone/Fork repo                        | ✅ Forked + cloned                       |
| 4. Run Python game                        | ✅ Done                                  |
| 5. Record 10 sec before                   | ✅ Done                                  |
| 6. Write prompts and fix broken code      | ⚠️ We need to account for this properly |
| 7. Add README features one by one         | ✅ Task 1 started; Tasks 2–4 still to do |
| 8. Record 10 sec after <b>all</b> changes | ⌚ Not yet — correct                     |
| 9. Push to personal repo                  | ⌚ Not yet                               |
| 10. Separate commits for each task        | ✅ Our plan is exactly this              |
| 11. Don't PR to SETAPESU26                | ✅ We won't                              |

The handout specifically requires the features to be added one by one and **separate commits for each task.**



Lab\_4\_VibeCoding\_Student\_handout

## The important part

We should **NOT** record the after-video yet.

That's actually correct because the after-video is supposed to be recorded **after all changes are finished**.  Lab\_4\_VibeCoding\_Student\_handout

And we should **not combine all four tasks into one commit**. Our approach of:

 Plain text

```
Task 1 → commit  
Task 2 → commit  
Task 3 → commit  
Task 4 → commit
```

is exactly what the assignment asks for.

## What about "fix the broken code"?

This is the only part we need to handle carefully.

We didn't invent a random bug just for the assignment. We used the Vibe Coding process to modify the supplied game and implement the README requirements. We can document our ChatGPT prompts/conversation as the Vibe Coding interaction, and the final deliverables include the exported chat history.  Lab\_4\_VibeCoding\_Student\_handout

**So don't undo anything.**

---

## Where we are right now

We have **Task 1 implemented locally**:

Multiple enemies — 3 enemies, stored in `self.enemies`, each independently updated with BFS.

We were at the point of committing it separately.

So let's simply continue from there.

**Run this now:**

```
</> powershell
```

```
git commit -m "Task 1: Add multiple enemies"
```

Send me the output.

Then I'll verify the commit, we'll push it, and we'll move to **Task 2**. We'll keep everything separated so that your Git history clearly demonstrates the four tasks.

are they saying I need to put everything into the forked repo?

Yes. **That is what the instructions mean.**

The important distinction is:

- **Assigned repo:** SETAPESU26/11\_maze\_chase → this is the original/main repo.
- **Your fork:** PES1UG24AM311/11\_maze\_chase-SE\_Lab---4- → this is where **your completed work should go**.
- **Local clone:** your folder on your PC → where you actually make and test the changes.

The instruction says:

"Push the code to your repo."  Lab\_4\_VibeCoding\_Student\_handout

Since you forked the assigned repository, **your fork is your repo**.

**So our final setup should look like this**

```
</> Plain text
```

Original repo

SETAPESU26/11\_maze\_chase

|

| fork

▼

Your repo

PES1UG24AM311/11\_maze\_chase-SE\_Lab---4-

▲

| git push

|

Your local project

C:\...\SE\Lab\Lab - 4\11\_maze\_chase-SE\_Lab---4-

And yes, the **completed code, all four tasks, commits, and final changes** should ultimately be

pushed to your fork.

## What should NOT go into the original repo?

Nothing needs to be pushed there.

The instruction:

**"Do not raise PR to the main repo (SETAPESU26)"**



Lab\_4\_VibeCoding\_Student\_handout

means **do not create a Pull Request from your fork back to SETAPESU26/11\_maze\_chase** .

So we're safe.

## And our four commits should end up in your fork

Something like:

Plain text

```
main
|
├─ Original repo commits
|
├─ Task 1: Add multiple enemies
|
├─ Task 2: Add speed up over time
|
├─ Task 3: Add power pellet
|
└─ Task 4: Add score by distance
```

Then your fork contains the complete finished project.

**One thing I would change from what we were about to do:** let's make sure the workflow is correct before pushing Task 1. Since you asked this at exactly the right time, we can verify the Git state first and then proceed.

Run:

powershell

```
git status
```

Send me the output.

the thing is.. they had said to put all labs in one single repo.. so I am confused

Ah — **that changes the interpretation**, and your confusion is completely reasonable.

There are potentially **two different repository requirements**:

1. **For Lab 4 itself**: the handout says to clone/fork the assigned repo, make the changes, and **push the code to your repo**.  Lab\_4\_VibeCoding\_Student\_handout
2. **For your course submission**: you were apparently told separately that **all SE labs must be maintained in one single personal repository**.

The Lab 4 handout you uploaded **does not mention that single-repository requirement**, so I don't want to pretend the handout says something it doesn't.  Lab\_4\_VibeCoding\_Student\_handout

**So we should pause before committing/pushing**

If your instructor's separate instruction is:

"Keep all Software Engineering labs in one repository"

then we need to determine **what they expect the Lab 4 folder structure to be**.

For example, it could be something like:

</> Plain text

```
SE-Lab/
|
├─ Lab-1/
├─ Lab-2/
├─ Lab-3/
├─ Lab-4/
|   └─ 11_maze_chase-SE_Lab---4-/
|       └─ game/
|           └─ main.py
|           └─ README.md
|           └─ requirements.txt
|
├─ before-video/
├─ after-video/
└─ chat-history.pdf
|
```

└ ...

But I don't want to assume that is their required structure.

**Most importantly: don't push Task 1 yet.**

We've only modified your local clone and haven't pushed the Task 1 commit. So **we have plenty of room to set this up correctly.**

If you have the instruction/message where your faculty said "**all labs in one single repo**", send me that (screenshot, text, or file).

I'll reconcile **that instruction + the Lab 4 handout** and tell you exactly:

- which repo should contain the code,
- whether your current fork is still needed,
- where Lab 4 should go,
- whether we should move the current work,
- and what the final Git/commit structure should look like.

**For now, don't commit or push anything.**

so for now, are we doing in that forked repo?

Yes. **For now, we are working in the forked repo.** ✓

Your current setup is:

Plain text

Original assigned repo  
SETAPESU26/11\_maze\_chase

↓ fork

Your fork  
PES1UG24AM311/11\_maze\_chase-SE\_Lab---4-

↓ clone

Your local folder  
C:\...\SE\Lab\Lab - 4\11\_maze\_chase-SE\_Lab---4-

So all the Task 1–4 code changes we're making right now are in **your forked repository.**

The only thing we're **not yet certain about** is whether, at the very end, your instructor wants this Lab 4 repo/code also copied into a larger **single repository containing all your SE labs.**

That does **not** affect what we're doing right now. We can finish the Lab 4 work in the fork, keep the four tasks as separate commits, and then decide how to incorporate it into the all-labs repository if required.

So for the immediate next step: **yes, stay in the forked repo.** 👍

And we haven't pushed Task 1 yet, so we're still in a safe position.

alright... we'll see what to do about pushing into the combined labs repo

Exactly. 👍

For now, we'll **finish Lab 4 properly in the forked repo** and keep the four tasks as separate commits.

Once everything is complete, we'll handle the **combined SE labs repository** separately so we don't accidentally mess up the Git history or submission structure.

So the immediate path remains:

1. ✅ Task 1 — Multiple Enemies
2. ⌚ Commit Task 1
3. ⌚ Push Task 1
4. Task 2 — Speed Up Over Time
5. Task 3 — Power Pellet
6. Task 4 — Score by Distance
7. Final testing
8. Record the **after** video
9. Export/save the ChatGPT conversation
10. Final push
11. Then decide how to place Lab 4 into the combined-labs repo.

And importantly, **we won't create a PR to SETAPESU26** .  Lab\_4\_VibeCoding\_Student\_handout

Let's continue from where we paused: **commit Task 1.**

I already committed.. I think README needs to be updated

Yes — **good catch.** Since Task 1 is already committed, we should update the README so it reflects the completed work and the remaining tasks.

But we should be careful: **I wouldn't rewrite the whole README yet.** The original README is also the specification for Tasks 2–4, so we should preserve those requirements and add a clear progress/completion section.

### For now

Since Task 1 is already committed, let's inspect your current README before changing it.

In your VS Code terminal, run:

```
</> powershell  
  
Get-Content README.md
```

Send me the output.

Then I'll tell you **exactly what to add/change**, and we'll make the README update as a **separate commit**, rather than mixing it into your Task 1 commit.



README.md-1.url  
File

just take like this bruv

Worked for 8s

Analyzed

Yep 😄 I get you now — you uploaded the **actual local README reference**, not asking me to make you paste it.

The uploaded file points to your local:

```
</> Plain text  
  
... \Lab - 4\11_maze_chase-SE_Lab---4-\README.md
```

I can work from the actual README, but the `.url` file only contains the **file path**, not the README contents itself. So I can't see the README text from that upload.

If you mean **“take the README as it currently is and update it to reflect the tasks we're completing”**, then yes — that's what we should do.

Since **Task 1 is already committed**, I'd suggest we **don't alter that commit**. We'll make the README update now as its own commit, then continue with Task 2.

If you want, just **upload the actual README.md file** (not the .url shortcut), and I'll give you the exact updated README to paste in. </>



README.md  
Document

Yep, now I can see the actual README. And **I would not rewrite it heavily**.

The README already contains the four required tasks and their prompts.  README

Since **Task 1 is completed**, the useful README change is simply to update the **Submission Checklist** at the bottom. Right now it still has all four tasks unchecked.  README

**Change this:**

</> Markdown

```
## Submission Checklist
```

- [ ] All 4 tasks completed
- [ ] Multiple enemies work independently
- [ ] Power pellet freezes enemy correctly
- [ ] Speed ramp increases difficulty over time
- [ ] Code reviewed with LLM (include chat link)

**To this for now:**

</> Markdown

```
## Submission Checklist
```

- [ ] All 4 tasks completed
- [x] Multiple enemies work independently
- [ ] Power pellet freezes enemy correctly
- [ ] Speed ramp increases difficulty over time
- [ ] Code reviewed with LLM (include chat link)

**That's it for now.**

Don't mark the other tasks complete until we actually implement and test them.

Also, because Task 1 is already committed, this README change would become a **separate commit**. That's perfectly fine; the requirement is to have separate commits for each task, and we shouldn't retroactively modify the Task 1 commit.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

If you're ready, I'll give you the **exact one-step command to make this README change**.

give

Yep. Let's do **only the README edit** first.

## Step 1 — Open README.md

In VS Code, open:

 Plain text

README.md

Go to the bottom and find:

 Markdown

## Submission Checklist

- ☐ All 4 tasks completed
- ☐ Multiple enemies work independently
- ☐ Power pellet freezes enemy correctly
- ☐ Speed ramp increases difficulty over time
- ☐ Code reviewed with LLM (include chat link)

## Step 2 — Change only the second checkbox

Change:

 Markdown

- ☐ Multiple enemies work independently

to:

⌕ Markdown

- [x] Multiple enemies work independently

So it becomes:

⌕ Markdown

## Submission Checklist

- [ ] All 4 tasks completed
- [x] Multiple enemies work independently
- [ ] Power pellet freezes enemy correctly
- [ ] Speed ramp increases difficulty over time
- [ ] Code reviewed with LLM (include chat link)

This matches what we've actually completed: Task 1 is done, while Tasks 2–4 aren't yet. The README's checklist explicitly separates these requirements.  README

## Step 3 — Save

Press:

⌕ Plain text

Ctrl + S

**Stop there. Don't commit yet.**

Tell me when you've saved it, and I'll give you the next step.

did it

Perfect. 👍

Now let's verify the README change before committing it.

Run:

⌕ powershell

```
git status
```

Send me the output. **Don't commit yet.**

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
Changes not staged for commit:
```

```
(use "git add <file>..." to update what will be committed)
```

```
(use "git restore <file>..." to discard changes in working  
directory)
```

```
    modified:   README.md
```

```
no changes added to commit (use "git add" and/or "git commit -  
a")
```

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4->
```

Perfect. That is exactly what we expected. 

It shows **only** `README.md` is **modified**, so your Task 1 commit remains untouched.

## Step 1 — Stage the README

Run:

```
</> powershell
```

```
git add README.md
```

Then run:

```
</> powershell
```

```
git status
```

Send me that output. **Don't commit yet.**

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git add README.md
```

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4->
```

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git status
```

On branch main

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

modified: README.md

Perfect.  Only README.md is staged.

## Step 2 — Commit the README update

Run:

```
</> powershell
```

```
git commit -m "Update README task progress"
```

Send me the output. **Don't push yet.**

done

Great. 

Now let's **push your current commits to your fork.**

Run:

```
</> powershell
```

```
git push origin main
```

This pushes your local Task 1 work **and** the README update to:

PES1UG24AM311/11\_maze\_chase-SE\_Lab---4-

It does **not** push anything to the original `SETAPESU26` repository.

Send me the output of `git push origin main`.

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4-> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 307 bytes | 307.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local
objects.
To https://github.com/PES1UG24AM311/11_maze_chase-
SE_Lab---4-.git
   a87f404..b6a814e  main -> main
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -
4\11_maze_chase-SE_Lab---4->
```

Perfect. 🎉 **Push succeeded.**

Your fork now contains:

- ☒ Task 1 — Multiple enemies
- ☒ README updated to mark Task 1 complete
- ☒ Separate commits
- ☒ Pushed to your fork
- ☒ Nothing pushed to `SETAPESU26`

## Next: Task 2 — Speed Up Over Time

We'll follow the README requirement exactly:

Every 15 seconds, reduce the enemy move interval by 2, with a minimum of 5, using `pygame.time.get_ticks()`, and show the current speed tier in the HUD.  README

And we'll do this **as a separate Task 2 commit**.

Before changing anything, I'll give you the Vibe Coding prompt we're using, then we'll make the

changes step-by-step and test them.

## Your Task 2 prompt

Paste this into our chat as the Vibe Coding prompt we're using:

"Add a difficulty ramp to my maze-chase game. Track elapsed time with `pygame.time.get_ticks()`. Every 15 seconds, reduce `enemy.move_interval` by 2 (minimum 5). Show current enemy speed tier in the HUD. The game now has multiple enemies stored in `self.enemies`, so the speed increase should apply consistently to all enemies."

Then we'll implement it.

**Don't change the code yet.** Tell me when you're ready, and we'll do Task 2 one step at a time.

let's do it

Let's do it. 👍

## Task 2 — Speed Up Over Time

We'll make the change in `game/game_engine.py`, because the game engine controls the elapsed game time and all three enemies.

The README requires:

- `pygame.time.get_ticks()`
- speed increase every **15 seconds**
- `move_interval` decreases by 2
- minimum interval 5
- speed tier shown in the HUD  [README](#)

## Step 1 — Open the file

In VS Code, open:

 Plain text

`game/game_engine.py`

Find your `reset()` method. You currently have something like:

</> Python

```
def reset(self):
    self.walls = generate_maze(COLS, ROWS)
    self.player = Player(0, 0)
    self.enemies = [
        Enemy(ROWS-1, COLS-1),
        Enemy(ROWS-1, 0),
        Enemy(0, COLS-1),
    ]
    self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
    self.caught = False
    self.won = False
```

Immediately **after the enemy list**, add:

</> Python

```
self.start_time = pygame.time.get_ticks()
self.speed_tier = 1
```

So it becomes:

</> Python

```
def reset(self):
    self.walls = generate_maze(COLS, ROWS)
    self.player = Player(0, 0)
    self.enemies = [
        Enemy(ROWS-1, COLS-1),
        Enemy(ROWS-1, 0),
        Enemy(0, COLS-1),
    ]
    self.start_time = pygame.time.get_ticks()
    self.speed_tier = 1
    self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
    self.caught = False
    self.won = False
```

**Do only this first.**

Save with `Ctrl + S` , then tell me **done**.

done

Perfect. 

## Step 2 — Add the speed-ramp calculation

Now find your `update()` method.

You currently have:

 Python

```
def update(self):
    if self.caught or self.won: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
```

Immediately **after** the `if self.caught or self.won: return` line, add:

 Python

```
elapsed = pygame.time.get_ticks() - self.start_time
self.speed_tier = 1 + elapsed // 15000
new_interval = max(5, 20 - (self.speed_tier - 1) * 2)

for enemy in self.enemies:
    enemy.move_interval = new_interval
```

So the beginning of `update()` should become:

 Python

```
def update(self):
    if self.caught or self.won: return

    elapsed = pygame.time.get_ticks() - self.start_time
    self.speed_tier = 1 + elapsed // 15000
    new_interval = max(5, 20 - (self.speed_tier - 1) * 2)

    for enemy in self.enemies:
        enemy.move_interval = new_interval

    keys = pygame.key.get_pressed()
```

```
self.player.move(keys, self.walls, ROWS, COLS)
```

## What this does

Plain text

```
0-14 sec      → Tier 1 → interval 20
15-29 sec     → Tier 2 → interval 18
30-44 sec     → Tier 3 → interval 16
45-59 sec     → Tier 4 → interval 14
...
105+ sec      → eventually reaches minimum interval 5
```

So **all three enemies speed up together**, which is appropriate now that we have `self.enemies` .

Save with:

Plain text

```
Ctrl + S
```

**Don't run the game yet.** Tell me when this part is done.

did it

Good. 

## Step 3 — Show the speed tier in the HUD

Now find this part near the bottom of `draw()` :

Python

```
info=self.font.render("Reach EXIT before the enemy catches you! R=Restart",True,(200,200,200))
self.screen.blit(info,(8,ROWS*CELL+14))
```

Replace those **two lines** with:

Python

```
info = self.font.render(
    f"Speed Tier: {self.speed_tier} | Survive! R=Restart",
```

```
True,  
(200, 200, 200)  
)  
self.screen.blit(info, (8, ROWS*CELL+14))
```

So your HUD will show something like:

 Plain text

```
Speed Tier: 1 | Survive! R=Restart
```

and after 15 seconds:

 Plain text

```
Speed Tier: 2 | Survive! R=Restart
```

This satisfies the README requirement to show the current enemy speed tier in the HUD.

 README

Save with:

 Plain text

```
Ctrl + S
```

Then **don't commit yet**. Tell me when you've done it.

did it

Perfect.  Task 2 code is now in place.

Before we commit anything, we need to **test it**.

## Step 4 — Run the game

In the terminal, make sure you're still in:

 Plain text

```
(.venv) ... \11_maze_chase-SE_Lab---4->
```

Run:

```
</> powershell
```

```
python main.py
```

Then check these things:

1. 🎮 Game opens normally.
2. 🦹 All 3 **enemies** appear.
3. HUD says:

```
</> Plain text
```

```
Speed Tier: 1 | Survive! R=Restart
```

4. Keep the game running for **at least 16–20 seconds**.
5. The HUD should change from:

```
</> Plain text
```

```
Speed Tier: 1
```

to:

```
</> Plain text
```

```
Speed Tier: 2
```

6. The game should continue running without errors.
7. Press **R** and verify the game resets and the tier goes back to 1.

You don't need to record anything yet — this is just testing.

When you're done, **close the game and send me what happened**, especially whether the tier changed to 2.

yes working

Excellent. ✅ **Task 2 is working correctly.**

Now we need to make sure the Task 2 changes are isolated and then commit them separately.

## Step 5 — Check the changes

Close the game if it is still open, then run:

```
</> powershell
```

```
git status
```

Send me the output.

**Don't commit yet.** I want to verify that the changes are only the files we expect before we create the Task 2 commit.

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git status  
On branch main  
Your branch is up to date with 'origin/main'.
```

```
Changes not staged for commit:  
  (use "git add <file>..." to update what will be committed)  
  (use "git restore <file>..." to discard changes in working  
directory)  
        modified:   game/game_engine.py
```

Perfect.  The status is exactly what we want:

```
</> Plain text
```

```
modified: game/game_engine.py
```

Only the Task 2 code is changed. Nothing else is being accidentally included.

## Step 6 — Stage Task 2

Run:

```
</> powershell
```

```
git add game/game_engine.py
```

Then:

```
</> powershell
```

```
git status
```

Send me the output. **Don't commit yet.**

```
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git add game/game_engine.py  
(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab -  
4\11_maze_chase-SE_Lab---4-> git status  
On branch main  
Your branch is up to date with 'origin/main'.
```

Changes to be committed:

```
(use "git restore --staged <file>..." to unstage)  
    modified:   game/game_engine.py
```

Perfect.  Only `game/game_engine.py` is staged.

## Step 7 — Commit Task 2

Run:

```
</> powershell
```

```
git commit -m "Task 2: Add speed ramp"
```

Send me the output of the commit command. **Don't push yet.**

did it

Perfect.  Task 2 is now committed separately.

Your Git history should now have:

- Task 1 — Multiple enemies

- README progress update
- Task 2 — Speed ramp

### Step 8 — Push Task 2 to your fork

Run:

```
</> powershell  
  
git push origin main
```

Send me the output. After I verify the push, we'll move on to **Task 3: Power Pellet (Freeze Enemy)**.

pushed

Great.  Task 2 is pushed successfully to your fork.

Current progress:

| Task                        | Status                                                                                                                     |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Task 1 — Multiple Enemies   |  Implemented, committed, pushed         |
| README checklist            |  Updated, committed, pushed             |
| Task 2 — Speed Up Over Time |  Implemented, tested, committed, pushed |
| Task 3 — Power Pellet       |  Next                                   |
| Task 4 — Score by Distance  |                                         |
| Final after-video           |                                         |
| Chat history export         |                                         |

### Task 3 — Power Pellet

The README specifies a **yellow circle somewhere in the maze**. When the player collides with it:

- `enemy.frozen = True`
- 300-frame countdown
- frozen enemies don't update

- frozen indicator displayed on the enemy  README

Because we now have **three enemies**, we'll adapt the requirement sensibly so that the pellet freezes **all enemies** when collected. This keeps the power-pellet mechanic consistent with the multi-enemy version.

We'll work mainly in:

 Plain text

```
game/entities.py
game/game_engine.py
```

And, as before, we'll make this a **separate Task 3 commit**.

## First step

Open:

 Plain text

```
game/entities.py
```

Find the `Enemy.__init__()` method. You currently have:

 Python

```
self.timer = 0
self.move_interval = 20
```

Immediately after those two lines, add:

 Python

```
self.frozen = False
self.freeze_timer = 0
```

So it becomes:

 Python

```
self.timer = 0
self.move_interval = 20
self.frozen = False
self.freeze_timer = 0
```

Only make this change for now. Save with `Ctrl + S` and tell me when done.

when did READme get updated for task 2?

Good catch — **we did not update the README for Task 2 yet.**

What happened was:

- After Task 1, we changed:

</> Plain text

[x] Multiple enemies work independently

- We committed that as:  
Update README task progress
- Then we completed Task 2 and committed:  
Task 2: Add speed ramp
- **But we never changed the README checkbox for the speed-ramp task.**

So right now your README should still show:

</> Plain text

```
[ ] All 4 tasks completed
[x] Multiple enemies work independently
[ ] Power pellet freezes enemy correctly
[ ] Speed ramp increases difficulty over time
[ ] Code reviewed with LLM (include chat link)
```

That's actually okay — **we caught it before moving further.**

**Let's fix it now**

Since Task 2 is already implemented, tested, committed, and pushed, we'll update only this line:

⌕ Markdown

```
- [ ] Speed ramp increases difficulty over time
```

to:

⌕ Markdown

```
- [x] Speed ramp increases difficulty over time
```

Then we'll make a separate README commit and push it.

This doesn't invalidate the Task 2 code commit. It just adds a documentation/progress commit afterward.

**So don't touch Task 3 yet.** Update that one checkbox first, save it, and tell me when done.

did it

Perfect. ✅

Now let's verify the README change before committing it.

Run:

⌕ powershell

```
git status
```

Send me the output. **Don't commit yet.**

yes it

Good. 🍌 Assuming `git status` shows only:

⌕ Plain text

```
Changes not staged for commit:
  modified: README.md
```

then stage it:

```
</> powershell  
git add README.md
```

Then verify:

```
</> powershell  
git status
```

Send me that output. **Don't commit yet.**

yes

Perfect. 👍 If `README.md` is now staged, commit this README update separately:

```
</> powershell  
git commit -m "Update README task progress"
```

Then **don't push yet**. Send me the commit output, and we'll verify it before pushing.

done

Great. ✅ The README progress update for Task 2 is now committed separately.

**Next — push that README commit**

Run:

```
</> powershell  
git push origin main
```

Send me the output.

After that, we'll continue with **Task 3: Power Pellet**.

yup

Nice. 🍌 Then Task 2's README update is pushed too.

Before we touch Task 3, let's keep the same disciplined workflow.

### Task 3 — Power Pellet

Open:

⟨/⟩ Plain text

game/entities.py

Find this in `Enemy.__init__()`:

⟨/⟩ Python

```
self.timer = 0
self.move_interval = 20
```

Add directly underneath:

⟨/⟩ Python

```
self.frozen = False
self.freeze_timer = 0
```

So it should be:

⟨/⟩ Python

```
self.timer = 0
self.move_interval = 20
self.frozen = False
self.freeze_timer = 0
```

Save with **Ctrl + S**.

**Don't make any other changes yet.** Tell me when you've done this, and we'll add the actual freeze behavior next.

done

Perfect. 

## Step 2 — Make the enemy actually freeze

Still in `game/entities.py`, find:

 Python

```
def update(self, walls, player, rows, cols):  
    from game.maze import bfs
```

We need the freeze countdown to happen before the normal BFS movement.

Replace the beginning of `update()` with:

 Python

```
def update(self, walls, player, rows, cols):  
    from game.maze import bfs  
  
    if self.frozen:  
        self.freeze_timer -= 1  
        if self.freeze_timer <= 0:  
            self.frozen = False  
    return
```

Then leave the existing code **after that** unchanged:

 Python

```
self.timer += 1  
if self.timer >= self.move_interval:  
    ...
```

So the complete method should start like this:

 Python

```
def update(self, walls, player, rows, cols):  
    from game.maze import bfs
```

```
if self.frozen:
    self.freeze_timer -= 1
    if self.freeze_timer <= 0:
        self.frozen = False
    return

self.timer += 1
if self.timer >= self.move_interval:
    ...
```

## Why

The README specifically requires that while frozen, `enemy.update()` does nothing and uses a **300-frame countdown**.  README

We're setting up the behavior now; the `300` value will be assigned when the player actually collects the pellet.

Save with **Ctrl + S**.

**Don't test yet.** Tell me when you've done this.

did it

Perfect. 

## Step 3 — Add the power pellet to the game

Now open:

 Plain text

game/game\_engine.py

Find the end of your `reset()` method. You currently have something like:

 Python

```
self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
self.caught = False
self.won = False
```

Add this **between** the `exit_rect` line and `self.caught` :

 Python

```
self.pellet_rect = pygame.Rect(
    (COLS//2)*CELL + CELL//2 - 8,
    (ROWS//2)*CELL + CELL//2 - 8,
    16,
    16
)
self.pellet_collected = False
```

So that section becomes:

 Python

```
self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
self.pellet_rect = pygame.Rect(
    (COLS//2)*CELL + CELL//2 - 8,
    (ROWS//2)*CELL + CELL//2 - 8,
    16,
    16
)
self.pellet_collected = False
self.caught = False
self.won = False
```

This gives us a **16×16 pellet** positioned around the center of the maze.

Save with:

 Plain text

Ctrl + S

**Don't run it yet.** Tell me when you've done this.

done

Good. 

## Step 4 — Freeze the enemies when the pellet is collected

Still in `game/game_engine.py` , find this part of `update()` :

</> Python

```
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
```

We need to check for pellet collection **before** updating the enemies.

Add this block immediately **before** the `for enemy in self.enemies:` loop:

</> Python

```
if not self.pellet_collected and self.player.rect.colliderect(self.pellet_rect):
    self.pellet_collected = True
    for enemy in self.enemies:
        enemy.frozen = True
        enemy.freeze_timer = 300
```

So that section becomes:

</> Python

```
if not self.pellet_collected and self.player.rect.colliderect(self.pellet_rect):
    self.pellet_collected = True
    for enemy in self.enemies:
        enemy.frozen = True
        enemy.freeze_timer = 300

for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
```

This means:

- Player touches pellet → pellet is collected.
- All three enemies become frozen.
- Each gets a **300-frame countdown**.
- `Enemy.update()` then handles the countdown and prevents movement while frozen.

This follows the README's 300-frame freeze requirement, adapted to your three-enemy version.

 README

Save with **Ctrl + S**.

**Don't run the game yet.** Tell me when you've done this.

done

Great. 

## Step 5 — Draw the power pellet

Now we need to make the pellet visible. Otherwise the player won't know where to collect it.

In `game/game_engine.py` , find:

 Python

```
self.player.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
```

Immediately **before** `self.player.draw(...)` , add:

 Python

```
if not self.pellet_collected:
    pygame.draw.circle(
        self.screen,
        (255, 220, 0),
        self.pellet_rect.center,
        8
    )
```

So it becomes:

 Python

```
if not self.pellet_collected:
    pygame.draw.circle(
        self.screen,
        (255, 220, 0),
```

```
        self.pellet_rect.center,  
        8  
    )  
  
    self.player.draw(self.screen)  
    for enemy in self.enemies:  
        enemy.draw(self.screen)
```

This gives us the **yellow circle** required by the README.  README

Save with **Ctrl + S**.

Then tell me when it's done.

done

Perfect. 

Now the power pellet is **created, collectible, and visible**.

## Step 6 — Show the frozen indicator

We need to modify `Enemy.draw()` so a frozen enemy visibly indicates that it's frozen, as required by the README.  README

Open:

 Plain text

game/entities.py

Find the end of `Enemy.draw()` :

 Python

```
for ex in [self.rect.x+4, self.rect.x+14]:  
    pygame.draw.circle(screen, (255,255,255), (ex, self.rect.y+8), 4)  
    pygame.draw.circle(screen, (0,0,0), (ex+1, self.rect.y+8), 2)
```

Immediately **after** that, add:

 Python

```
if self.frozen:
    pygame.draw.circle(
        screen,
        (100, 200, 255),
        self.rect.center,
        16,
        3
    )
```

So the end of `draw()` becomes:

 Python

```
for ex in [self.rect.x+4, self.rect.x+14]:
    pygame.draw.circle(screen, (255,255,255), (ex, self.rect.y+8), 4)
    pygame.draw.circle(screen, (0,0,0), (ex+1, self.rect.y+8), 2)

if self.frozen:
    pygame.draw.circle(
        screen,
        (100, 200, 255),
        self.rect.center,
        16,
        3
    )
```

This will put a visible blue ring around a frozen enemy.

Save with **Ctrl + S**.

Then we'll run the game and test the entire Task 3 mechanic.

done

Perfect. 

## Step 7 — Test Task 3

Now let's test the complete power-pellet mechanic.

Run:

 powershell

python main.py

Check:

1. 🟡 A **yellow pellet** appears near the center of the maze.
2. 🤖 All 3 enemies appear normally.
3. Move the player onto the yellow pellet.
4. The pellet should disappear.
5. All 3 enemies should show the **blue frozen indicator**.
6. The enemies should stop moving.
7. After roughly **5 seconds** (300 frames at 60 FPS), the blue indicators disappear and the enemies start moving again.
8. Make sure the game doesn't crash.
9. Press **R** and verify the pellet comes back and the game resets.

**Important:** The pellet is placed at the maze center, which is also where the existing EXIT is located. So when testing, you may see the player reach the exit at the same time as the pellet depending on the generated maze. If that happens, tell me—we'll adjust the pellet location to a safe maze cell rather than guessing.

Run the test and tell me exactly what happens.

Hello from the pygame community. <https://www.pygame.org/contribute.html>

Traceback (most recent call last):

File "C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11\_maze\_chase-SE\_Lab---4-\main.py", line 5, in <module>  
engine.run()

File "C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11\_maze\_chase-SE\_Lab---4-\game\game\_engine.py", line 141,  
in run

self.draw()

File "C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11\_maze\_chase-SE\_Lab---4-\game\game\_engine.py", line 100,  
in draw

for ex in [self.rect.x+4, self.rect.x+14]:

^^^^^^^^

AttributeError: 'GameEngine' object has no attribute 'rect'. Did you mean: 'reset'?

(.venv) PS C:\Personal\PES College Related\Sem - 5\SE\Lab\Lab - 4\11\_maze\_chase-SE\_Lab---4->

Worked for 7s

The error is straightforward:

</> Plain text

```
AttributeError: 'GameEngine' object has no attribute 'rect'
```

`self.rect` belongs to an **Enemy**, not `GameEngine` .

We need to move the code we just added to the correct method.

### Step 1 — Fix `game/game_engine.py`

Open:

</> Plain text

`game/game_engine.py`

In `draw()` , you currently have the accidental block:

</> Python

```
for ex in [self.rect.x+4, self.rect.x+14]:
    pygame.draw.circle(screen, (255,255,255), (ex, self.rect.y+8), 4)
    pygame.draw.circle(screen, (0,0,0), (ex+1, self.rect.y+8), 2)

if self.frozen:
    pygame.draw.circle(
        screen,
        (100, 200, 255),
        self.rect.center,
        16,
        3
    )
```

Delete that entire block.

Do not delete this correct part:

Python

```
for enemy in self.enemies:
    enemy.draw(self.screen)
```

So the relevant portion of `GameEngine.draw()` should simply be:

Python

```
self.player.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
```

## Step 2 — Put the indicator in the correct file

Now open:

Plain text

`game/entities.py`

Go to the **bottom** of `Enemy.draw()` .

It should already contain:

Python

```
def draw(self, screen):
    pygame.draw.rect(screen, self.color, self.rect, border_radius=5)
    for ex in [self.rect.x+4, self.rect.x+14]:
        pygame.draw.circle(screen, (255,255,255), (ex, self.rect.y+8), 4)
        pygame.draw.circle(screen, (0,0,0), (ex+1, self.rect.y+8), 2)
```

Add **only this** after that:

Python

```
if self.frozen:
    pygame.draw.circle(
        screen,
        (100, 200, 255),
```

```
        self.rect.center,  
        16,  
        3  
    )
```

So `Enemy.draw()` ends as:

 Python

```
def draw(self, screen):  
    pygame.draw.rect(screen, self.color, self.rect, border_radius=5)  
    for ex in [self.rect.x+4, self.rect.x+14]:  
        pygame.draw.circle(screen, (255,255,255), (ex, self.rect.y+8), 4)  
        pygame.draw.circle(screen, (0,0,0), (ex+1, self.rect.y+8), 2)  
  
    if self.frozen:  
        pygame.draw.circle(  
            screen,  
            (100, 200, 255),  
            self.rect.center,  
            16,  
            3  
        )
```

**Then save both files.**

**Don't run the game yet.**

Tell me when you've made those two corrections, and I'll have you run it again.

done

Great. 👍

Now let's test again.

## Step — Run Task 3

Run:

 powershell

```
python main.py
```

Then check:

1. 🟡 Yellow pellet appears.
2. 🐙 Three enemies appear.
3. Move the player onto the pellet.
4. Pellet disappears.
5. All three enemies get the blue frozen ring.
6. They stop moving.
7. After about **5 seconds**, they unfreeze and move again.

Also check that the **Speed Tier HUD still works**.

If it runs successfully, tell me what you observed. If there's another traceback, paste it exactly like before and we'll fix it before committing anything.

yellow pellet is stuck on exit

Exactly — that's because we placed the pellet at the **same center position as the EXIT**. Good catch. We should move it to a different maze cell.

## Fix the pellet position

Open:

📄 Plain text

game/game\_engine.py

Find this:

📄 Python

```
self.pellet_rect = pygame.Rect(
    (COLS//2)*CELL + CELL//2 - 8,
    (ROWS//2)*CELL + CELL//2 - 8,
    16,
    16
)
```

Replace it with:

Python

```
self.pellet_rect = pygame.Rect(
    1 * CELL + CELL//2 - 8,
    1 * CELL + CELL//2 - 8,
    16,
    16
)
```

This places the pellet at **row 1, column 1**, away from the center EXIT.

So that section should be:

Python

```
self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
self.pellet_rect = pygame.Rect(
    1 * CELL + CELL//2 - 8,
    1 * CELL + CELL//2 - 8,
    16,
    16
)
self.pellet_collected = False
self.caught = False
self.won = False
```

Save with **Ctrl + S**.

Then run:

powershell

```
python main.py
```

Now the pellet should appear in the **second row, second column**, rather than on the EXIT.

Don't commit yet. Test whether you can reach the pellet and whether the freeze mechanic works.

should the pellet disappear after one time I hit it?

Yes. It should disappear after the player collects it once. 

That's how we've implemented it:

 Python

```
if not self.pellet_collected and self.player.rect.colliderect(self.pellet_rect):  
    self.pellet_collected = True
```

And the drawing code says:

 Python

```
if not self.pellet_collected:  
    pygame.draw.circle(...)
```

So the behavior is:

1. 🟡 Pellet appears.
2. 🧑 Player touches it.
3. `pellet_collected` becomes `True`.
4. 🟡 Pellet disappears.
5. 🤖 All 3 enemies freeze for **300 frames (~5 seconds at 60 FPS)**.
6. After 300 frames, they unfreeze.
7. The pellet **does not come back during that game**.
8. Pressing **R** resets the game, so the pellet appears again.

That's consistent with the README's requirement to place a power pellet and have collecting it freeze the enemy for 5 seconds.  [README](#)

So if yours disappears after the first collision, **that's correct**.

Is there anything mentioned about position of pellet having to change every game? I don't want to do extra but need to meet expectation of task

No. The README does not say that the pellet position has to change every game.

The actual Task 3 requirement says:

"Place a power pellet in the maze. Collecting it freezes the enemy for 5 seconds."  [README](#)

And its prompt only specifies:

- a yellow circle **somewhere in the maze**
- collision with the player
- `enemy.frozen = True`
- 300-frame countdown
- frozen enemy doesn't update
- frozen indicator

It **does not require random placement**, changing position between games, or multiple pellets.

So our fixed position at row 1, column 1 is sufficient. You don't need to add randomization or anything extra just to meet the assignment.

One small thing we **did** need to change was moving it away from the EXIT, because putting both at the same location makes the mechanic awkward to demonstrate. That's a practical correction, not an extra feature.

**Keep it fixed.** No need to over-engineer Task 3. 👍

ok nice

👍 Good. We'll keep Task 3 **exactly to the README requirement** and avoid unnecessary extras.

Next, finish testing the pellet:

- 🟡 Collect it once
- 🐙 All 3 enemies freeze
- 🔵 Frozen indicator appears
- ⌚ They unfreeze after ~5 seconds
- 🟡 Pellet stays gone until R resets the game

Once you've confirmed that works, **tell me**, and we'll move to the next step: checking the Task 3 changes before committing them.

it's working

Excellent. ✅ **Task 3 is working.**

Before we commit it, let's verify that only the intended Task 3 files changed.

## Step 1 — Check Git status

Close the game, then run:

```
</> powershell  
git status
```

Send me the output.

**Don't stage or commit yet.**

yes

Great. 👍 Since you confirmed the game is working, let's check the Git state now.

Run:

```
</> powershell  
git status
```

Send me the **exact output**. I want to verify that only the Task 3 files are modified before we stage anything.

yes verified

Perfect. 👍

Since you've verified the Git status and Task 3 is working, let's stage the two files involved in Task 3:

```
</> powershell  
git add game/entities.py game/game_engine.py
```

Then verify:

```
</> powershell  
git status
```

Send me the output. **Don't commit yet.**

[next](#)

Yep — next step is to **stage the Task 3 files**.

Run:

```
</> powershell  
  
git add game/entities.py game/game_engine.py
```

Then:

```
</> powershell  
  
git status
```

You should see:

```
</> Plain text  
  
Changes to be committed:  
  modified: game/entities.py  
  modified: game/game_engine.py
```

Send me that output, and then we'll create the **Task 3 commit**.

[next](#)

Next, **commit Task 3**.

Run:

```
</> powershell  
  
git commit -m "Task 3: Add power pellet freeze"
```

Send me the output. **Don't push yet**.

[then](#)

After the Task 3 commit, the next step is to **push it to your fork**.

Run:

```
</> powershell  
  
git push origin main
```

You should see something like:

```
</> Plain text  
  
main -> main
```

Once that succeeds, **don't start Task 4 immediately**. We should first update the README checkbox for Task 3, just like we did for Tasks 1 and 2, then push that README update.

So the sequence is:

1.  Commit Task 3
2.  Push Task 3
3.  Mark **Power pellet freezes enemy correctly** as [x] in README
4.  Commit README update
5.  Push README update
6.  Start Task 4

Go ahead with:

```
</> powershell  
  
git push origin main
```

and tell me when it's done.

done

Perfect.  Task 3 is now pushed to your fork.

Now let's update the README to reflect that Task 3 is complete.

**Step 1 — Open README.md**

Find:

⌕ Markdown

- ☐ Power pellet freezes enemy correctly

Change it to:

⌕ Markdown

- ☒ Power pellet freezes enemy correctly

So your checklist should now be:

⌕ Markdown

## Submission Checklist

- ☐ All 4 tasks completed
- ☒ Multiple enemies work independently
- ☒ Power pellet freezes enemy correctly
- ☒ Speed ramp increases difficulty over time
- ☐ Code reviewed with LLM (include chat link)

Save with:

⌕ Plain text

Ctrl + S

Then tell me when you've done it.

done this

Perfect. 

Now verify the README change:

⌕ powershell

```
git status
```

You should see only:

```
</> Plain text
```

```
modified: README.md
```

Send me the output. **Don't commit yet.**

next

Next, commit the README update.

Run:

```
</> powershell
```

```
git add README.md
git commit -m "Update README task progress"
```

Then **don't push yet**. Send me the commit output, and we'll verify it before pushing.

next

Next, push the README commit:

```
</> powershell
```

```
git push origin main
```

Once that succeeds, we'll start **Task 4 — Score by Distance**.

done

Excellent.  **Task 3** and its README update are both pushed.

We can now move to the final feature.

## Task 4 — Score by Distance

The README requires:

- Increase `self.score` by 1 every frame while the player is alive
- Display `Survived: Xs` in the HUD, where `X = score // 60`
- Show the final score on the game-over overlay  [README](#)

We'll keep this strictly to those requirements.

## Step 1 — Add the score variable

Open:

 Plain text

`game/game_engine.py`

Inside `reset()`, near:

 Python

```
self.caught = False
self.won = False
```

add:

 Python

```
self.score = 0
```

So the end of `reset()` should look like:

 Python

```
self.pellet_collected = False
self.caught = False
self.won = False
self.score = 0
```

Save with **Ctrl + S**.

**Don't make any other change yet.** Tell me when you've done this.

[next](#)

Next. 👍

## Step 2 — Increment the score every frame

In the same `game/game_engine.py` , find the beginning of `update()` :

Python

```
def update(self):  
    if self.caught or self.won: return
```

Immediately after that line, add:

Python

```
self.score += 1
```

So it becomes:

Python

```
def update(self):  
    if self.caught or self.won: return  
    self.score += 1  
  
    elapsed = pygame.time.get_ticks() - self.start_time
```

This means the score increases by **1 each frame while the player is still alive**, matching the README requirement. [📖 README](#)

Save with **Ctrl + S**.

Don't make the HUD change yet. Tell me when done.

did it

done

done

yup works

next

next

next

next

yup

next

done

took video

ya nothing

</> entities.py  
Code

</> game\_engine.py  
Code

did

did

done

is there no better way?

give docx

Analyzed

</>

I think they definitely want full chat history

so pdf is the only way huh

do it as docx

but it says it'll take days

nvm let's do pdf